

- 6장: 객체와 메소드에 대한 추가 내용
 - 생성자: **new** 키워드 사용할 때 자동으로 호출되는 메소드
 - 여러개의 생성자를 가질 수 있다. → 생성자 오버로딩
 - 인스턴스 변수: 객체마다 고유한 값을 가지는 변수
 - 객체의 상태를 나타냄
 - **private**로 선언하여 정보 은닉을 구현함
 - **static** 변수: 클래스의 모든 객체들이 공유하는 변수
 - 클래스 변수라고 불리며 클래스당 하나만 존재
 - **static**과 **final**을 사용하면 상수가 되어 값 변경 불가
 - **static** 메소드: 특정 객체에 종속되지 않고 클래스 이름을 사용하여 호출 할 수 있는 메소드
 - **static**변수에 접근, 객체의 상태와 무관한 작업을 할 때
 - **static**메소드에선 인스턴스 변수나 일반 메소드를 참조 할 수 없음
 - 오버로딩: 동일한 메서드 이름이지만 서로 다른 시그니처를 갖는다.
 - 시그니처는 매개변수 개수, 타입, 순서의 조합
 - 정보 은닉: 객체 내부 구현 세부 사항을 숨긴다.
 - **private**와 **public**으로 구분 대부분 **private**로 외부접근으로 막고 **getter**와 **setter**로 접근시킴, 이는 캡슐화 시킬 수 있음
- 7장 배열
 - 배열: 동일한 데이터 타입의 여러 데이터 값들을 하나의 이름으로 묶어 관리하는 데이터 구조
 - 첨자 혹은 인덱스로 정수값을 사용하여 개별적으로 식별하고 접근
 - 원시타입보다는 복잡하지만 객체보다 간단한특징을 가짐
 - 첨자 표기법을 사용함
 - 첨자/인덱스: 배열 내에서 특정 요소의 위치를 나타내는 정수값
 - 0부터 시작하는 0넘버링
 - 마지막 요소는 **length - 1**
 - 배열 이름 뒤에 대괄호 안에 표기
 - 길이: 배열의 인스턴스 변수
 - 배열이 가질 수 있는 최대요소의 개수
 - **length**는 **final**로 선언되어있으므로 배열길이는 변환 할 수 없음
 - 배열요소, 첨자변수
 - 배열이름과 첨자를 함께 사용하여 특정 위치에 있는 단일 데이터 값
 - 첨자변수 자체는 일반 변수처럼 사용 될 수 있으며, 메소드의 매개변수로 전달가능
 - 부분적으로 채워진 배열
 - 전체 용량 중 일부만 실제 데이터로 채워져 있는 상태의 배열
 - 실제 유효한 데이터의 개수를 추적하기 위해 보통 별도의 정수변수를 함께사용
 - 순차 탐색: 배열에서 특정한 값을 찾기 위한 기본적인 탐색 알고리즘
 - 배열의 시작부터 순서대로 각 요소를 확인하여 찾으려는 값과 비교
 - 값을 발견하거나 배열의 끝에 도달 할 때 까지 탐색을 계속함
 - 간단하지만 효율성은 떨어집니다.

- 8장 상속과 다형성
 - 상속: 기존 클래스의 변수와 메소드를 새로운 클래스가 물려받는 개념
 - **extends** 키워드로 구현
 - 파생클래스는 기본 클래스의 **public** 및 **protected** 멤버에 접근 할 수 있음
 - 기본 클래스 / 파생 클래스
 - 기본클래스는 상속을 통해 물려주는 클래스
 - 슈퍼클래스, 부모클래스 라고도 합니다.
 - 파생 클래스는 물려받는 클래스
 - 파생 클래스, 자식클래스라고 합니다.
 - 파생클래스는 기본 클래스의 기능을 재사용하고 확장하여 새로운 기능을 추가하거나 기존 기능을 오버라이딩 할 수 있습니다.
 - 클래스 계층 구조
 - 클래스 계층 구조는 상속관계로 연결된 클래스들의 트리 구조를 의미함
 - 가장 상위에는 하나 이상의 기본 클래스가 있고 그 아래로 여러 단계의 파생 클래스들이 이어 질 수 있습니다.
 - 오버라이딩
 - 파생클래스가 기본클래스에 정의된 메소드와 동일한 이름과 시그니처를 가지는 메소드를 재정의 하는 것을 의미합니다.
 - 오버라이딩된 메소드는 객체의 실제 타입에 따라 실행되며 다형성의 핵심 요소 중 하나입니다.
 - 기본 클래스의 오버라이딩 되기 전 메소드를 호출해야 할 때는 **super** 키워드를 사용합니다.
 - 다형성 및 동적 바인딩
 - 다형성은 하나의 객체가 여러 타입으로 취급 될 수 있는 객체 지향 프로그래밍의 특징
 - 특히 기본 클래스 타입의 참조 변수가 파생클래스 타입의 객체를 참조 할 때 나타남
 - 동적바인딩은 다형성의 한 형태로 메소드 호출이 런타임에 객체의 실제 타입에 따라 결정되는 것을 의미합니다.
 - 예를들어 기본클래스 참조변수가 파생클래스 객체를 가리킬 때 오버라이딩된 메소드를 호출하면 파생클래스의 메소드가 실행됩니다.
 - 인터페이스
 - 메소드의 선언만 포함하고 구현은 포함하지 않는 추상타입
 - **implements** 키워드를 사용하여 하나 이상의 인터페이스를 구현할 수 있으며 인터페이스에 선언된 모든 메소드를 반드시 구현해야 합니다.
 - 인터페이스는 다형성을 위한 또 다른 메커니즘을 제공하며 클래스들이 따라야 할 계약을 정의하는데 사용됩니다.
- 9장 예외처리
 - 예외처리
 - 프로그램 실행 중 발생 할 수 있는 비정상적인 상황을 감지하고 복구하여 프로그램이 비정상적으로 종료되는 것을 방지한 프로그래밍 기법

- 정상적인 코드와 예외상황을 처리하는 코드를 분리하여 프로그램의 가독성과 유지보수성을 향상시킵니다.
 - **try, catch, throw, finally** 키워드를 사용하여 구현합니다.
 - **try block**
 - 예외가 발생 할 가능성이 있는코드를 포함하는 부분
 - 예외가 발생하면 즉시 **catch**블록으로 제어가 이동됨
 - **catch block**
 - **try**블록에서 발생한 예외를 처리하는 코드를 포함하는 부분
 - **catch**블록은 특정 예외타입을 매개변수로 받아 해당 타입의 예외가 발생했을 때 실행됩니다.
 - 예외 객체를 통해 예외에 대한 정보를 얻을 수 있습니다.
 - 하나의 **try** 블록은 여러개의 **catch**블록을 가질 수 있습니다.
 - **throw문**
 - **throw**문은 프로그램 실행 중에 명시적으로 예외 객체를 생성하고 발생시키는 데 사용됩니다.
 - 예외 객체를 생성하고 예외를 던집니다.
 - **throw**문이 실행되고면 현재 메소드의 실행이 중단되고 해당 예외를 처리할 수 있는 **catch**블록이나 **throws** 절을 가진 호출 스택 상위의 메소드로 제어가 넘어갑니다.
 - **throws 절**
 - **throws**절은 메소드 선언부에 추가하여 해당 메소드 내에서 발생할 수 있는 예외의 타입을 호출자에게 알리는 역할을 합니다.
 - 메소드내에서 발생한 예외를 직접 **try-catch** 블록으로 처리하지 않고 예외처리 책임을 해당 메소드를 호출하는 쪽으로 넘길 때 사용합니다.
 - **Checked 예외**
 - 컴파일시점에서 확인되는 예외로 **try-catch**나 **throws**로 선언해야 함
 - **Finally 블록**
 - **try-catch**블록 뒤에 오며 예외 발생 유무와 관계없이 항상 실행이 보장되는 코드를 포함합니다.
 - 일반적으로 파일스트림, 네트워크 연결과 같은 리소스를 닫거나 해제하는 코드를 작성함으로서 리소스 누수를 방지합니다.
- **10장**
 - **텍스트 파일 입출력**
 - 문자 데이터를 파일로 읽고 쓰는 작업입니다.
 - **PrintWriter class**를 사용하여 텍스트 파일에 출력하고 **Scanner** 클래스를 사용하여 텍스트 파일에서 입력을 읽습니다.
 - **PrintWriter**
 - 텍스트 데이터를 출력 스트림으로 보내는 데 사용되는 필터 클래스
 - **print, println**등의 메소드 제공 원시타입 값이나 문자열 쓰게 도와줌
 - 파일이름이나 다른 출력 스트림을 인수로 제공합니다.
 - **close**로 출력 스트림을 닫아야 합니다.
 - **Scanner**
 - 키보드 입력 뿐 아니라 파일입력에도 사용 할 수 있습니다.
 - **File**객체나 **InputStream**을 인수로 제공합니다.
 - **File class**
 - 파일 시스템의 경로명을 객체로 표현하는 방법
 - **append**

- `printWriter`나 `File output stream`으로 파일을 열면 기존 파일의 내용을 덮어씀
 - 기존 파일 내용 끝에 새로운 데이터를 추가하고 싶을 때 **append** 모드를 사용함
- **close** 메소드
 - 파일 입출력 스트림을 사용한 후 스트림을 닫아야 함
 - 출력스트림은 **close**해야 버퍼에 남아있는 데이터가 기록됨
- **CSV**파일
 - 콤마같은 구분자로 되어있는 파일 **split**으로 나눌 수 있다
- 이진 파일 입출력
 - 텍스트 형식이 아닌 컴퓨터가 이해하는 이진 파일에 직접 읽고 쓰는 작업
- **WriteObject** or **Read Object**
 - 객체를 이진형식으로 파일에 쓰고 읽는데 사용됨
 - 객체를 직렬화 하고 역직렬화 하려면 객체의 클래스가 **Serializable**인터페이스를 구현해야함
- **Serializable** 인터페이스
 - 객체들을 이진 파일로 저장하거나 네트워크로 전송하기 위해 직렬화 할 수 있음을 표시하느는 마커 인터페이스